Rapidez - Moto Uber Carora

Documentação Técnica Completa de Arquitetura, Backend e Aplicações Móveis

---

1. Visão Geral e Contexto de Carora

O **Rapidez** é uma plataforma móvel e web de mobilidade urbana sob demanda (estilo Uber), projetada especificamente para operar no transporte de passageiros por mototáxi na cidade de Carora, Estado Lara, Venezuela.

Contexto Local e Mapeamento

Em Carora, a infraestrutura urbana e a convenção de navegação dependem fortemente de referências populares e pontos de embarque bem conhecidos em detrimento do uso exclusivo de nomes formais de ruas e números residenciais.

- **Pontos de Referência Comuns:** Hospital Central San Juan Bautista, Plaza Bolívar, Terminal de Pasajeros de Carora, Avenida Francisco de Miranda, Zona Industrial, Centro Histórico e estabelecimentos comerciais chave.
- **Mapeamento Flexível:** O sistema permite a seleção geográfica exata via coordenadas (latitude e longitude) acompanhada obrigatoriamente por um campo de texto descritivo de "Ponto de Referência" (ex: *En frente de la bodega de Don Pedro, casa portón blanco*).

Dinâmica Monetária e Multimoeda

Devido ao cenário econômico local, a plataforma suporta liquidação e exibição de tarifas em regime multimoeda simultâneo:

- **Moeda Base:** Dólar Americano (USD).
- **Moeda Local de Liquidação:** Bolívar Soberano (VES), convertido em tempo real via taxa de referência informada pelo backend (taxa oficial ou de mercado predefinida).
- **Pago Móvil (Interbancário):** Método de pagamento eletrônico primário na Venezuela. O passageiro realiza a transferência via aplicativo do seu banco inserindo o Cédula de Identidad (ID), Telefone e Banco do motorista, informando os últimos 4 a 6 dígitos do número de referência da transação no aplicativo.
- **Espécie (Efectivo):** Suporte a pagamentos em papel moeda (USD ou VES exacto/troco).

---

2. Arquitetura do Sistema e Stack Tecnológica

O ecossistema é baseado em uma arquitetura distribuída e orientada a eventos em tempo real para sincronização contínua de geolocalização e estados das viagens.+----------+

| CAMADA DE CLIENTE |

| +----------+ +----------+ |

| | App Passageiro (Flutter) | | App Mototaxista (Flutter) | |

| | - Mapbox/Google Maps UI | | - Background Location | |

| | - State Management (Bloc/Provider)| | - Screen Lock Overlay | |

| +----------+ +----------+ |

+----------+----------+

| HTTP / WebSockets (Socket.io)

v

+----------+

| CAMADA DE APLICAÇÃO |

| +----------+ |

| | Node.js & Express API | |

| | - REST Endpoints (Autenticação, Histórico, Perfis) | |

| | - Socket.io Gateway (Rastreamento, Despacho, Geo-Events) | |

| | - FCM Engine (Push Notifications via Firebase Admin SDK) | |

| +----------+ |

+----------+----------+

|

+----------+----------+

| |

v v

+----------------------------------+ +-----------------------------+

| CAMADA DE DADOS | | CAMADA DE CACHE & PUB/SUB|

| PostgreSQL 15 + Extension PostGIS| | Redis Server 7 |

| - Persistência Relacional | | - Lat/Lng em memória (GEO) |

| - Consultas Espaciais (ST_DWithin| | - Estado de Socket Activo |

+----------------------------------+ +-----------------------------+

Componentes da Stack

- **Frontend Móvel:** Flutter (Dart) para builds nativos cross-platform (Android & iOS).
- **Backend Runtime:** Node.js (v18+ LTS) com Express.js para APIs RESTful.
- **Motor em Tempo Real:** Socket.io para comunicação duplex bidirecional.
- **Banco de Dados Principal:** PostgreSQL 15 com a extensão espacial **PostGIS** ativada.
- **Cache Espacial e Mensageria em Memória:** Redis 7 (utilizando estruturas de dados `GEO` e Pub/Sub).
- **Push Notifications:** Firebase Cloud Messaging (FCM) via Firebase Admin SDK backend.

---

3. Infraestrutura e Docker Compose

O ambiente de desenvolvimento e produção do backend e serviços associados é orquestrado via Docker Compose.version: '3.8'

services:

  postgres:

    image: postgis/postgis:15-3.3

    container_name: rapidez_postgres

    restart: always

    environment:

      POSTGRES_DB: rapidez_db

      POSTGRES_USER: rapidez_admin

      POSTGRES_PASSWORD: SecurePassword123!

```
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U rapidez_admin -d rapidez_db"]
      interval: 10s
      timeout: 5s
      retries: 5
  redis:
    image: redis:7-alpine
    container_name: rapidez_redis
    restart: always
    command: redis-server --requirepass RedisSecurePass123! --appendonly yes
    ports:
      - "6379:6379"
    volumes:
      - redisdata:/data
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
volumes:
```

```
pgdata:

  driver: local

redisdata:

  driver: local
```

---

4. Esquema de Banco de Dados SQL (PostgreSQL + PostGIS)

O modelo relacional suporta geolocalização nativa utilizando o tipo espacial `GEOMETRY(Point, 4326)` (WGS 84).l

```
-- Ativação da extensão PostGIS
CREATE EXTENSION IF NOT EXISTS postgis;

-- Tabela de Passageiros
CREATE TABLE passengers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
full_name VARCHAR(100) NOT NULL,
phone_number VARCHAR(20) UNIQUE NOT NULL,
email VARCHAR(100) UNIQUE,
fcm_token TEXT,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Tabela de Motoristas (Mototaxistas)
CREATE TABLE drivers (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
full_name VARCHAR(100) NOT NULL,
phone_number VARCHAR(20) UNIQUE NOT NULL,
national_id VARCHAR(20) UNIQUE NOT NULL, -- Cédula de Identidad
vehicle_plate VARCHAR(10) NOT NULL,
vehicle_model VARCHAR(50) DEFAULT 'Bera X1000',
is_active BOOLEAN DEFAULT false,
is_available BOOLEAN DEFAULT false,
current_location GEOMETRY(Point, 4326),
pago_movil_bank_code VARCHAR(10),
pago_movil_phone VARCHAR(20),
fcm_token TEXT,
rating NUMERIC(3,2) DEFAULT 5.00,
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Índice Espacial GiST para otimização de busca geográfica de motoristas
CREATE INDEX idx_drivers_location ON drivers USING GIST (current_location);
```

```
-- Tabela de Viagens (Trips)
CREATE TYPE trip_status AS ENUM (
'REQUESTED',
'ACCEPTED',
'DRIVER_ARRIVED',
'IN_PROGRESS',
'COMPLETED',
'CANCELLED'
);

CREATE TYPE payment_method AS ENUM ('CASH_USD', 'CASH_VES', 'PAGO_MOVIL');
CREATE TYPE payment_status AS ENUM ('PENDING', 'VERIFYING', 'PAID', 'FAILED');

CREATE TABLE trips (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
passenger_id UUID NOT NULL REFERENCES passengers(id),
driver_id UUID REFERENCES drivers(id),
status trip_status DEFAULT 'REQUESTED',-- Localização de Origem

origin_location GEOMETRY(Point, 4326) NOT NULL,

origin_address TEXT NOT NULL,

origin_reference TEXT NOT NULL, -- Ponto de referência em Carora

-- Localização de Destino

destination_location GEOMETRY(Point, 4326) NOT NULL,

destination_address TEXT NOT NULL,

destination_reference TEXT,

-- Financeiro e Pagamento

fare_usd NUMERIC(10,2) NOT NULL,

exchange_rate_ves NUMERIC(10,2) NOT NULL,

payment_method payment_method NOT NULL,

payment_status payment_status DEFAULT 'PENDING',

payment_reference_code VARCHAR(50),

created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
```

```
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP

);

-- Índice Espacial na origem e destino das viagens
CREATE INDEX idx_trips_origin ON trips USING GIST (origin_location);

-- Consulta Espacial Exemplo: Encontrar motoristas disponíveis em um raio de 3km em Carora (ST_DWithin)
-- Nota: 4326 usa graus, converte-se para geography para buscar em metros exatos.
SELECT
id,
full_name,
phone_number,
ST_Y(current_location::geometry) as latitude,
ST_X(current_location::geometry) as longitude,
ST_Distance(current_location::geography, ST_SetSRID(ST_MakePoint(-70.0827, 10.1755), 4326)::geography) as distance_meters
FROM drivers
WHERE is_active = true
AND is_available = true
AND ST_DWithin(
current_location::geography,
ST_SetSRID(ST_MakePoint(-70.0827, 10.1755), 4326)::geography,
3000 -- Raio de 3000 metros (3km)
)
ORDER BY distance_meters ASC;
```

---

5. Backend em Tempo Real (Node.js, Socket.io e FCM)

O serviço de backend processa atualizações de posição dos motoristas via Socket.io, calcula a proximidade em relação ao ponto de embarque utilizando a fórmula de Haversine e envia alertas Push via FCM quando a distância for menor que 100 metros.

```javascript
// server.js

const express = require('express');

const http = require('http');

const { Server } = require('socket.io');
```

```
const admin = require('firebase-admin');
const { Pool } = require('pg');
const Redis = require('ioredis');
const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });
const redis = new Redis({ host: 'localhost', port: 6379, password: 'RedisSecurePass123!' });
const db = new Pool({
  user: 'rapidez_admin',
  host: 'localhost',
  database: 'rapidez_db',
  password: 'SecurePassword123!',
  port: 5432,
});
// Inicialização Firebase Admin SDK
// serviceAccountKey.json deve ser providenciado pelo console Firebase
const serviceAccount = require('./serviceAccountKey.json');
admin.initializeApp({
  credential: admin.credential.cert(serviceAccount)
});
// Função utilitária: Cálculo da fórmula de Haversine em Metros
function calculateHaversineDistance(lat1, lon1, lat2, lon2) {
  const R = 6371e3; // Raio da Terra em metros
  const phi1 = (lat1 * Math.PI) / 180;
```

```
  const phi2 = (lat2 * Math.PI) / 180;
  const deltaPhi = ((lat2 - lat1) * Math.PI) / 180;
  const deltaLambda = ((lon2 - lon1) * Math.PI) / 180;
  const a =
    Math.sin(deltaPhi / 2) * Math.sin(deltaPhi / 2) +
    Math.cos(phi1) * Math.cos(phi2) * Math.sin(deltaLambda / 2) * Math.sin(deltaLambda / 2);
  const c = 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
  return R * c; // Distância em metros
}

// Gerenciamento de conexões Socket.io
io.on('connection', (socket) => {
  console.log(`Cliente conectado: ${socket.id}`);

  // Registro na sala individual
  socket.on('join_room', ({ userId, role }) => {
    socket.join(`user_${userId}`);
    console.log(`Usuário ${userId} (${role}) entrou na sala user_${userId}`);
  });

  // Atualização de localização em tempo real do Mototaxista
  socket.on('driver_location_update', async (data) => {
    const { driverId, tripId, latitude, longitude, passengerId } = data;

    // 1. Atualizar posição no Redis (GEO)
    await redis.geoadd('drivers_locations', longitude, latitude, driverId);

    // 2. Transmitir localização para o passageiro em tempo real
    if (tripId && passengerId) {
```

```
      io.to(`user_${passengerId}`).emit('driver_position_changed', {
        driverId,
        latitude,
        longitude,
      });
      // 3. Verificar proximidade do embarque se houver uma viagem ativa
      const tripQuery = await db.query(
        'SELECT ST_Y(origin_location::geometry) as orig_lat, ST_X(origin_location::geometry) as orig_lng
FROM trips WHERE id = $1',
        [tripId]
      );
      if (tripQuery.rows.length > 0) {
        const { orig_lat, orig_lng } = tripQuery.rows[0];
        const distanceMeters = calculateHaversineDistance(latitude, longitude, orig_lat, orig_lng);
        // Se a distância for menor que 100m e o alerta ainda não foi emitido
        if (distanceMeters <= 100) {
          const alertSent = await redis.get(`alert_sent:${tripId}`);
          if (!alertSent) {
            await triggerProximityPushNotification(passengerId, distanceMeters);
            await redis.set(`alert_sent:${tripId}`, 'true', 'EX', 1800); // Expirar em 30 min
          }
        }
      }
    }
```

```
  });

  socket.on('disconnect', () => {

    console.log(`Cliente desconectado: ${socket.id}`);

  });

});

// Envio de Notificação Push via FCM

async function triggerProximityPushNotification(passengerId, distance) {

  try {

    const userResult = await db.query('SELECT fcm_token FROM passengers WHERE id = $1',
[passengerId]);

    const fcmToken = userResult.rows[0]?.fcm_token;

    if (!fcmToken) return;

    const payload = {

      token: fcmToken,

      notification: {

        title: '¡Tu mototaxi está cerca!',

        body: `El conductor está a menos de ${Math.round(distance)} metros de tu punto de encuentro.`,

      },

      data: {

        click_action: 'FLUTTER_NOTIFICATION_CLICK',

        type: 'DRIVER_PROXIMITY',

      },

    };

    await admin.messaging().send(payload);

    console.log(`Notificação FCM de proximidade enviada para o usuário ${passengerId}`);
```

```
  } catch (error) {
    console.error('Erro ao enviar FCM:', error);
  }
}

server.listen(3000, () => {
  console.log('Servidor Backend Rapidez rodando na porta 3000');
});
```

---

6. Aplicativo Flutter do Passageiro

O aplicativo do passageiro apresenta uma interface intuitiva centralizada no mapa, permitindo definir a localização através de um *Pin* central e especificando explicitamente referências locais de Carora.

Componente de Mapa e Interpolação de Animação Smooth

```
t
import 'dart:async';
import 'dart:ui';
import 'package:flutter/material.dart';
import 'package:google_maps_flutter/google_maps_flutter.dart';
import 'package:socket_io_client/socket_io_client.dart' as IO;

class PassengerMapScreen extends StatefulWidget {
final String passengerId;
const PassengerMapScreen({Key? key, required this.passengerId}) : super(key: key);

@override
State createState() => _PassengerMapScreenState();
}

class _PassengerMapScreenState extends State {
final Completer _mapController = Completer();
final TextEditingController _referenceController = TextEditingController();

LatLng _centerPosition = const LatLng(10.1755, -70.0827); // Carora Centro
Marker? _driverMarker;
IO.Socket? _socket;
```

```
@override
void initState() {
super.initState();
_initSocketConnection();
}

void _initSocketConnection() {
_socket = IO.io('http://10.0.2.2:3000', <String, dynamic>{
'transports': ['websocket'],
'autoConnect': false,
});
_socket!.connect();_socket!.onConnect((_) {

  _socket!.emit('join_room', {'userId': widget.passengerId, 'role': 'PASSENGER'});

});

// Escutar mudança de posição do motorista em tempo real

_socket!.on('driver_position_changed', (data) {

  double newLat = data['latitude'];

  double newLng = data['longitude'];

  _animateDriverMarker(LatLng(newLat, newLng));

});

}

// Animação de movimento suave usando interpolação linear (lerp)
void _animateDriverMarker(LatLng targetPosition) {
if (_driverMarker == null) {
setState(() {
_driverMarker = Marker(
markerId: const MarkerId('driver'),
position: targetPosition,
icon: BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueYellow),
);
});
return;
}LatLng startPosition = _driverMarker!.position;

int startTime = DateTime.now().millisecondsSinceEpoch;

int durationMs = 2000; // 2 segundos de interpolação
```

```
Timer.periodic(const Duration(milliseconds: 16), (timer) {
  int elapsed = DateTime.now().millisecondsSinceEpoch - startTime;
  double t = elapsed / durationMs;
  if (t > 1.0) t = 1.0;
  // Interpolação Linear de Latitude e Longitude
  double lat = lerpDouble(startPosition.latitude, targetPosition.latitude, t)!;
  double lng = lerpDouble(startPosition.longitude, targetPosition.longitude, t)!;
  setState(() {
    _driverMarker = Marker(
      markerId: const MarkerId('driver'),
      position: LatLng(lat, lng),
      icon: BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueYellow),
    );
  });
  if (t >= 1.0) {
    timer.cancel();
  }
});
}

@override
Widget build(BuildContext context) {
return Scaffold(
body: Stack(
children: [
GoogleMap(
initialCameraPosition: CameraPosition(target: _centerPosition, zoom: 15),
onMapCreated: (controller) => _mapController.complete(controller),
```

```
onCameraMove: (position) {
_centerPosition = position.target;
},
markers: _driverMarker != null ? {_driverMarker!} : {},
),     // Pin Fixo Central para Seleção de Origem/Destino

    const Center(

      child: Padding(

        padding: EdgeInsets.only(bottom: 35.0),

        child: Icon(Icons.location_on, size: 45, color: Colors.redAccent),

      ),

    ),

    // Painel Inferior de Entrada de Referência de Carora

    Positioned(

      bottom: 0, left: 0, right: 0,

      child: Container(

        padding: const EdgeInsets.all(16),

        decoration: const BoxDecoration(

          color: Colors.white,

          borderRadius: BorderRadius.vertical(top: Radius.circular(20)),

          boxShadow: [BoxShadow(blurRadius: 10, color: Colors.black12)],

        ),

        child: Column(

          mainAxisSize: MainAxisSize.min,

          children: [

            TextField(

              controller: _referenceController,
```

```
            decoration: const InputDecoration(
              labelText: 'Punto de Referencia en Carora',
              hintText: 'Ej: Frente a la farmacia, casa azul...',
              prefixIcon: Icon(Icons.storefront),
              border: OutlineInputBorder(),
            ),
          ),
          const SizedBox(height: 12),
          ElevatedButton(
            style: ElevatedButton.styleFrom(
              backgroundColor: Colors.amber[700],
              minimumSize: const Size.fromHeight(50),
            ),
            onPressed: () {
              // Disparar criação de corrida enviando _centerPosition e _referenceController.text
            },
            child: const Text('CONFIRMAR PUNTO DE RECOGIDA', style: TextStyle(color: Colors.black)),
          ),
        ],
      ),
    ),
  ],
),
```

);

}

@override
void dispose() {
_socket?.disconnect();
_socket?.dispose();
super.dispose();
}
}

---

7. Aplicativo Flutter do Mototaxista

O aplicativo do condutor executa um serviço Foreground em segundo plano (`flutter_background_service`) que mantém o envio constante do GPS mesmo se o dispositivo estiver bloqueado ou com a tela desligada.

Serviço em Segundo Plano e Tela de Alerta

```dart

import 'dart:async';

import 'dart:ui';

import 'package:flutter/material.dart';

import 'package:flutter_background_service/flutter_background_service.dart';

import 'package:geolocator/geolocator.dart';

import 'package:socket_io_client/socket_io_client.dart' as IO;

Future<void> initializeBackgroundService() async {

  final service = FlutterBackgroundService();

  await service.configure(

    androidConfiguration: AndroidConfiguration(

      onStart: onStartBackgroundService,

      autoStart: false,
```

```
      isForegroundMode: true,
      notificationTitle: "Rapidez Conductor Activo",
      notificationContent: "Transmitiendo ubicación en segundo plano...",
      initialNotificationTitle: "Rapidez",
      initialNotificationContent: "Iniciando servicio de rastreo...",
      foregroundServiceNotificationId: 888,
    ),
    iosConfiguration: IosConfiguration(
      autoStart: false,
      onForeground: onStartBackgroundService,
    ),
  );
}

@pragma('vm:entry-point')
void onStartBackgroundService(ServiceInstance service) async {
  DartPluginRegistrant.ensureInitialized();
  IO.Socket socket = IO.io('http://10.0.2.2:3000', <String, dynamic>{
    'transports': ['websocket'],
    'autoConnect': true,
  });
  // Configuração das opções de precisão do Geolocator
  LocationSettings locationSettings = const LocationSettings(
    accuracy: LocationAccuracy.high,
    distanceFilter: 5, // Atualiza a cada 5 metros movidos
```

```
  );

  Geolocator.getPositionStream(locationSettings: locationSettings).listen((Position position) {

    socket.emit('driver_location_update', {

      'driverId': 'DRIVER_UUID_HERE',

      'tripId': 'CURRENT_TRIP_UUID_IF_ANY',

      'latitude': position.latitude,

      'longitude': position.longitude,

    });

  });

  service.on('stopService').listen((event) {

    service.stopSelf();

  });

}
```

---

8. Fluxo e Validação de Pagamento via Pago Móvil

O fluxo financeiro via Pago Móvil exige rigor na transferência de dados para evitar erros de digitação e garantir rápida validação no local.

Esquema do Fluxo

```
[Passageiro]                [Servidor Backend]                [Mototaxista]

    |                             |                             |

    |--- 1. Solicita Dados de Pago ----->|                             |

    |<-- 2. Retorna Telefone, ID, Banco -|                             |

    |                             |                             |

    | (Realiza Pago Móvil no App do Banco)                              |

    |                             |                             |
```

|--- 3. Envia Ref (ex: 481920) ---->| |

| Status: VERIFYING |--- 4. Notifica Notificação ----->|

| | "Pago Recibido por Validar" |

| | |

| |<-- 5. Motorista Valida/Confirma -|

|<-- 6. Notifica Viagem Concluída ---| Status: PAID |

Tabela de Mapeamento de Bancos (Venezuela)

Código Banco	Nome da Instituição Financeira
0102	Banco de Venezuela (BDV)
0108	Banco Provincial
0105	Banco Mercantil
0134	Banesco
0114	Banco del Tesoro

Implementação do Modal de Pagamento no Flutter

t

```
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

class PagoMovilDialog extends StatelessWidget {
final String phone = "04125551234";
final String nationalId = "V18999888";
final String bankName = "Banco de Venezuela (0102)";
final double amountVes = 185.50;

final TextEditingController _refController = TextEditingController();

PagoMovilDialog({Key? key}) : super(key: key);

void _copyToClipboard(BuildContext context, String text, String label) {
Clipboard.setData(ClipboardData(text: text));
ScaffoldMessenger.of(context).showSnackBar(
SnackBar(content: Text('$label copiado al portapapeles')),
```

```
);
}

@override
Widget build(BuildContext context) {
return AlertDialog(
title: const Text('Pago Móvil Rapidez', style: TextStyle(fontWeight: FontWeight.bold)),
content: SingleChildScrollView(
child: Column(
crossAxisAlignment: CrossAxisAlignment.start,
mainAxisSize: MainAxisSize.min,
children: [
Text('Monto a transferir: $amountVes VES', style: const TextStyle(fontWeight: FontWeight.bold,
fontSize: 16, color: Colors.green)),
const Divider(),
ListTile(
contentPadding: EdgeInsets.zero,
title: Text('Teléfono: $phone'),
trailing: IconButton(icon: const Icon(Icons.copy), onPressed: () => _copyToClipboard(context, phone,
'Teléfono')),
),
ListTile(
contentPadding: EdgeInsets.zero,
title: Text('Cédula/RIF: $nationalId'),
trailing: IconButton(icon: const Icon(Icons.copy), onPressed: () => _copyToClipboard(context,
nationalId, 'Cédula')),
),
ListTile(
contentPadding: EdgeInsets.zero,
title: Text('Banco: $bankName'),
trailing: IconButton(icon: const Icon(Icons.copy), onPressed: () => _copyToClipboard(context, '0102',
'Código Banco')),
),
const SizedBox(height: 10),
TextField(
controller: _refController,
keyboardType: TextInputType.number,
maxLength: 6,
decoration: const InputDecoration(
labelText: 'Número de Referencia (Últimos 6 dígitos)',
border: OutlineInputBorder(),
),
),
],
),
),
```

```
actions: [
TextButton(onPressed: () => Navigator.pop(context), child: const Text('CANCELAR')),
ElevatedButton(
style: ElevatedButton.styleFrom(backgroundColor: Colors.green),
onPressed: () {
if (_refController.text.length >= 4) {
// Notificar backend sobre a referência do Pago Móvil
Navigator.pop(context, _refController.text);
}
},
child: const Text('REPORTAR PAGO', style: TextStyle(color: Colors.white)),
),
],
);
}
}
```

---

9. Guia Passo a Passo de Execução e Despliegue

Pré-requisitos

* Docker e Docker Compose instalados no servidor de destino.

* Node.js v18+ e NPM instalados.

* SDK do Flutter configurado localmente para os builds do app.

* Chave de Conta de Serviço Firebase (`serviceAccountKey.json`).

Passo 1: Subir Infraestrutura de Banco de Dados e Cache

```bash

# Iniciar contêineres do PostgreSQL/PostGIS e Redis em modo desacoplado

docker-compose up -d

# Verificar se os serviços estão com status healthy

docker-compose ps
```

Passo 2: Configuração e Execução do Backend Node.js

```
# Navegar até o diretório do backend

cd backend
```

Instalar dependências da aplicação

npm install

Iniciar o servidor em modo de produção

npm start

Passo 3: Build e Distribuição dos Aplicativos Flutter

Build do Aplicativo do Passageiro (Android APK)

cd apps/passenger_app

flutter clean

flutter pub get

flutter build apk --release --build-name=1.0.0 --build-number=1

Build do Aplicativo do Mototaxista (Android APK com permissões estendidas)

cd apps/driver_app

flutter clean

flutter pub get

flutter build apk --release --build-name=1.0.0 --build-number=1

Conclusão e Observações Finais

A arquitetura proposta garante alto desempenho e baixa latência no envio de dados de localização, suportando conexões simultâneas via Socket.io com custo computacional otimizado graças ao suporte do Redis. As particularidades geográficas e financeiras de Carora são atendidas integralmente pela inclusão de referências customizadas no PostGIS e suporte nativo ao fluxo de Pago Móvil.